Masked k-NN Imputation with missknn

Distance masking, per-column tuning, and donor-pool capping for mixed tabular data.

Imad EL BADISY

2026-08-24

Table of contents

Mixed tabular data with missing values, $X = (x_{ij}) \in \mathbb{R}^{n \times p}$, is usually imputed with an iterative random-forest engine such as `missForest`, which needs no distributional assumptions but scales poorly with n and can fail outright when a rare categorical level is entirely absent from a training fold. `missknn` targets the same mixed numeric and categorical setting with a k-nearest-neighbor engine instead, built so that distances only ever compare jointly observed variables, and so that neighborhood size and estimator are chosen per column by internal holdout evaluation rather than fixed in advance.[1]

Masked distance

Let $R_{ij} = \mathbb{1}[x_{ij} \text{ is observed}]$ and $O_i = \{j : R_{ij} = 1\}$. For a receiver row i with a missing target t, and a donor row ℓ with $R_{\ell t} = 1$, the shared observed set excluding the target itself is

$$S_{i\ell t} = O_i \cap O_\ell \setminus \{t\}.$$

[1] `missknn` package documentation, CRAN: https://CRAN.R-project.org/package=missknn.

Pairs with $S_{i\ell t} = \emptyset$ share nothing comparable and are discarded. Numeric columns are standardized from observed values only, $z_{ij} = (x_{ij} - \mu_j)/\sigma_j$, and each shared variable contributes

$$\delta_j(i,\ell) = \begin{cases} (z_{ij} - z_{\ell j})^2 & j \text{ numeric} \\ \mathbb{1}[x_{ij} \neq x_{\ell j}] & j \text{ categorical.} \end{cases}$$

The masked distance is a weighted rate over the shared observed mass, not a sum discounted by how much is missing:

$$d_t(i,\ell) = \sqrt{\frac{\sum_{j \in S_{i\ell t}} w_j \, \delta_j(i,\ell)}{\sum_{j \in S_{i\ell t}} w_j}},$$

with weights $w_j \geq 0$, uniform by default. Normalizing by $\sum_{j \in S_{i\ell t}} w_j$ rather than a fixed $p - 1$ means a donor pair with less overlap is not automatically penalized: distance measures how different two rows are on what is actually comparable between them, the same role partial matching plays in Gower's coefficient.[2]

Neighbor aggregation

Given $d_t(i, \cdot)$, let $N_k(i,t)$ be the k donors with smallest finite distance. For a numeric target, the default estimator is an inverse-distance-weighted mean,

$$\hat{x}_{it} = \frac{\sum_{\ell \in N_k(i,t)} \alpha_{i\ell t} \, x_{\ell t}}{\sum_{\ell \in N_k(i,t)} \alpha_{i\ell t}}, \qquad \alpha_{i\ell t} = \frac{1}{d_t(i,\ell) + \varepsilon},$$

and for a categorical target a weighted-majority vote, $\hat{x}_{it} = \arg\max_v \sum_{\ell \in N_k(i,t)} \alpha_{i\ell t} \mathbb{1}[x_{\ell t} = v]$. Under multiple imputation ($m > 1$), each completed dataset draws a donor from $N_k(i,t)$ with probability proportional to $\alpha_{i\ell t}$ instead of collapsing to the point estimate, so imputation uncertainty is propagated rather than repeated m times.

Local regression

A neighbor average ignores how the target co-varies with other predictors. `missknn` also fits a local ridge regression over a wider neighborhood $N_{k'}(i,t)$, $k' > k$:

$$\hat{\beta} = \arg\min_{\beta} \sum_{\ell \in N_{k'}(i,t)} \left(x_{\ell t} - \beta^\top \mathbf{z}_\ell \right)^2 + \lambda \|\beta_{-0}\|_2^2,$$

[2] Gower, J. C. (1971). A general coefficient of similarity and some of its properties. *Biometrics*, 27(4), 857-871.

where $\mathbf{z}_\ell$ stacks the intercept with observed numeric predictors and the ridge penalty λ leaves the intercept unregularized. Neighbors are weighted uniformly here rather than by inverse distance: a near-duplicate donor is a stabilizing dominant vote for a mean, but the same near-duplicate makes the regression's normal equations nearly singular, so the weighting scheme that helps the mean estimator would actively hurt the regression one.

Per-column tuning

Neither k, k', nor the choice between the mean and the regression estimator is fixed globally. For each column, `missknn` holds out roughly 25 percent of that column's observed entries, capped at 100 rows, imputes them from the remaining donors under each candidate k, and scores holdout error. An improvement is only accepted once it clears a fixed relative margin over the running best, 8 percent for neighborhood size and 20 percent for switching to regression, because with finite holdout data a smaller k or a regression fit can look marginally better than the incumbent by chance alone, and the larger margin for regression reflects that it is being compared as the better of two noisy candidates rather than a single alternative.

The same holdout scoring also evaluates the plain global mean or mode of the column. If nothing clears the margin against this baseline, the column is filled directly from it and neighbor search is skipped entirely, both because a k-point average carries needless variance when a column has no local signal to exploit, and because the global fill costs $O(1)$ per receiver rather than a full neighbor search. For columns that do need neighbor search, an unbounded search across all donors is $O(n_{\text{recv}} \times n_{\text{don}})$. When the donor pool exceeds a cap (2000 by default), a random subsample of that size is drawn once per column and reused for every receiver, keeping cost linear in the number of receivers rather than quadratic in n.

```
Require: X, R, weights w, candidate grid K, donor cap C
D <- donors of column t;  M <- receivers of column t
if |D| = 0: fill M with a constant default; return
(H, T) <- random holdout/train split of D
e* <- holdout error of global mean/mode;  k* <- null;  est* <- global
for k in K:
    e_k <- holdout error of the weighted mean at neighborhood k
    if e_k < e*(1 - 0.08): e* <- e_k; k* <- k; est* <- mean
k'_narrow <- max(4 k*, 30);  k'_wide <- min(n_train, 300)
e_reg <- best holdout error of the ridge regression at k'_narrow, k'_wide
if e_reg < e*(1 - 0.20): est* <- regression
if est* = global: fill M from the global mean/mode of D; return
D' <- random subsample of D of size min(|D|, C)
for i in M: rank D' by masked distance; fill x_it via the mean or the regression estimator
```

This procedure runs once per column on the first pass, and once more per completed dataset when $m > 1$, with stochastic donor draws in place of the point estimate.

Example

The `missknn` package exposes a single constructor with a `complete()` accessor.

```
library(missknn)

set.seed(1)
x <- data.frame(
  a = c(rnorm(20, 10, 2)),
  b = c(rnorm(20, 5, 1)),
  g = factor(sample(c("low", "mid", "high"), 20, replace = TRUE))
)
x$a[sample(20, 4)] <- NA
x$b[sample(20, 3)] <- NA
x$g[sample(20, 2)] <- NA

imp <- missknn(x, k = 5, m = 1, numeric_estimator = "regression", seed = 1)
imp
```

```
<missknn>
  observations: 20
  variables: 3
  k: 5
  completed datasets: 1
  scale: TRUE
  add_indicator: FALSE
```

Nine cells were missing across the three columns (4 in `a`, 3 in `b`, 2 in `g`), single imputation was requested (`m = 1`), and the printed summary confirms one completed dataset was produced from a 20-row, 3-column input.

The completed data replaces every missing entry with its estimate.

```
out <- as.data.frame(complete(imp))
out$a <- round(out$a, 2)
out$b <- round(out$b, 2)
head(out, 8)
```

```
      a    b    g
1  8.75 5.92  mid
2 10.37 5.78  low
```

```
3  8.33 5.07 high
4 13.19 3.01  low
5 10.66 5.62  low
6  8.36 4.94  low
7 11.20 5.68  low
8 11.48 3.53 high
```

Every row shown is now fully observed; rows that originally had a missing `a`, `b`, or `g` value (not visible here, since the printed rows happen to be complete ones) received an estimate from their nearest donors under the masked distance rather than being dropped.

`summary()` reports how much was missing per column and the settings used for the fit.

```
str(summary(imp))
```

```
List of 8
 $ n                 : int 20
 $ p                 : int 3
 $ k                 : int 5
 $ m                 : int 1
 $ scale             : logi TRUE
 $ add_indicator     : logi FALSE
 $ missing_by_variable: Named int [1:3] 4 3 2
  ..- attr(*, "names")= chr [1:3] "a" "b" "g"
 $ missing_total     : int 9
 - attr(*, "class")= chr "summary.missknn"
```

`missing_by_variable` reproduces the 4/3/2 split noted above, and `missing_total` confirms the 9 missing cells overall; `k`, `scale`, and `add_indicator` echo back the settings this particular fit used, useful for confirming what was actually run rather than what was intended.

Benchmark: synthetic data (reproducing the submitted paper's Table 1)

The evaluation design below is the one used in the submitted paper and shipped as `inst/benchmark/benchmark_30.R` in the package: 30 simulations of $y = \beta^\top x + \varepsilon$ with five independent predictors ($n = 200$), 20 percent MCAR amputation on the predictors via `mimar::ampute()`, comparing `missknn` against `missForest`, `missRanger`, and `VIM::kNN` on runtime, imputation accuracy (NRMSE, pooled across every imputed entry), and downstream OLS coefficient error, following the original `missForest` evaluation design.

```r
library(mimar)
library(missForest)
library(missRanger)
library(VIM)
library(ggplot2)

METHODS <- c("missknn", "missForest", "missRanger", "VIM::kNN")

simulate_data <- function(n = 200L, p = 5L) {
  x <- replicate(p, rnorm(n), simplify = FALSE)
  names(x) <- paste0("x", seq_len(p))
  x <- as.data.frame(x)
  beta <- c(0.6, -0.4, 0.35, 0.25, -0.2)
  eta <- 0.5 + as.matrix(x) %*% beta
  y <- as.numeric(eta + rnorm(n, sd = 0.75))
  list(data = data.frame(y = y, x, check.names = FALSE), beta = beta)
}

nrmse <- function(true_vals, imp_vals) {
  v <- stats::var(true_vals)
  if (!is.finite(v) || v <= 0) return(NA_real_)
  sqrt(mean((true_vals - imp_vals)^2) / v)
}
```

```r
set.seed(20260705)
R <- 30
runtime_rows <- list(); nrmse_rows <- list(); coef_rows <- list()

for (s in seq_len(R)) {
  sim <- simulate_data()
  amp <- mimar::ampute(sim$data[, -1], prop = 0.2, mechanism = "MCAR",
                       target = names(sim$data)[-1], seed = s)
  x_miss <- amp$data
  truth_x <- sim$data[, setdiff(names(sim$data), "y"), drop = FALSE]
  na_mask <- as.data.frame(is.na(x_miss))

  t0 <- Sys.time(); c_knn <- missknn::complete(missknn(x_miss, k = 5L, m = 1L, seed = s)); rt
  t0 <- Sys.time(); c_mf  <- missForest::missForest(x_miss, verbose = FALSE)$ximp; rt_mf <- 
  t0 <- Sys.time(); c_mr  <- missRanger::missRanger(x_miss, verbose = 0, num.trees = 100, nu
  t0 <- Sys.time(); c_vim <- as.data.frame(VIM::kNN(x_miss, k = 5L, imp_var = FALSE)); rt_vi

  completed <- list(missknn = c_knn, missForest = c_mf, missRanger = c_mr, `VIM::kNN` = c_vi
```

```
  runtimes  <- c(missknn = rt_knn, missForest = rt_mf, missRanger = rt_mr, `VIM::kNN` = rt_v

  for (m in METHODS) {
    imp <- completed[[m]]
    true_pooled <- unlist(lapply(names(truth_x), function(j) truth_x[[j]][na_mask[[j]]]))
    imp_pooled  <- unlist(lapply(names(truth_x), function(j) imp[[j]][na_mask[[j]]]))
    nrmse_rows[[length(nrmse_rows) + 1]] <- data.frame(sim = s, method = m, nrmse = nrmse(tr

    completed_full <- data.frame(y = sim$data$y, imp, check.names = FALSE)
    beta_hat <- stats::coef(stats::lm(y ~ ., data = completed_full))
    common <- intersect(names(sim$data)[-1], names(beta_hat))
    diff <- beta_hat[common] - sim$beta[match(common, paste0("x", seq_along(sim$beta)))]
    coef_rows[[length(coef_rows) + 1]] <- data.frame(sim = s, method = m, abs_bias = mean(ab
  }
  runtime_rows[[s]] <- data.frame(sim = s, method = METHODS, runtime = unname(runtimes))
}

runtime_df <- do.call(rbind, runtime_rows)
nrmse_df   <- do.call(rbind, nrmse_rows)
coef_df    <- do.call(rbind, coef_rows)
```

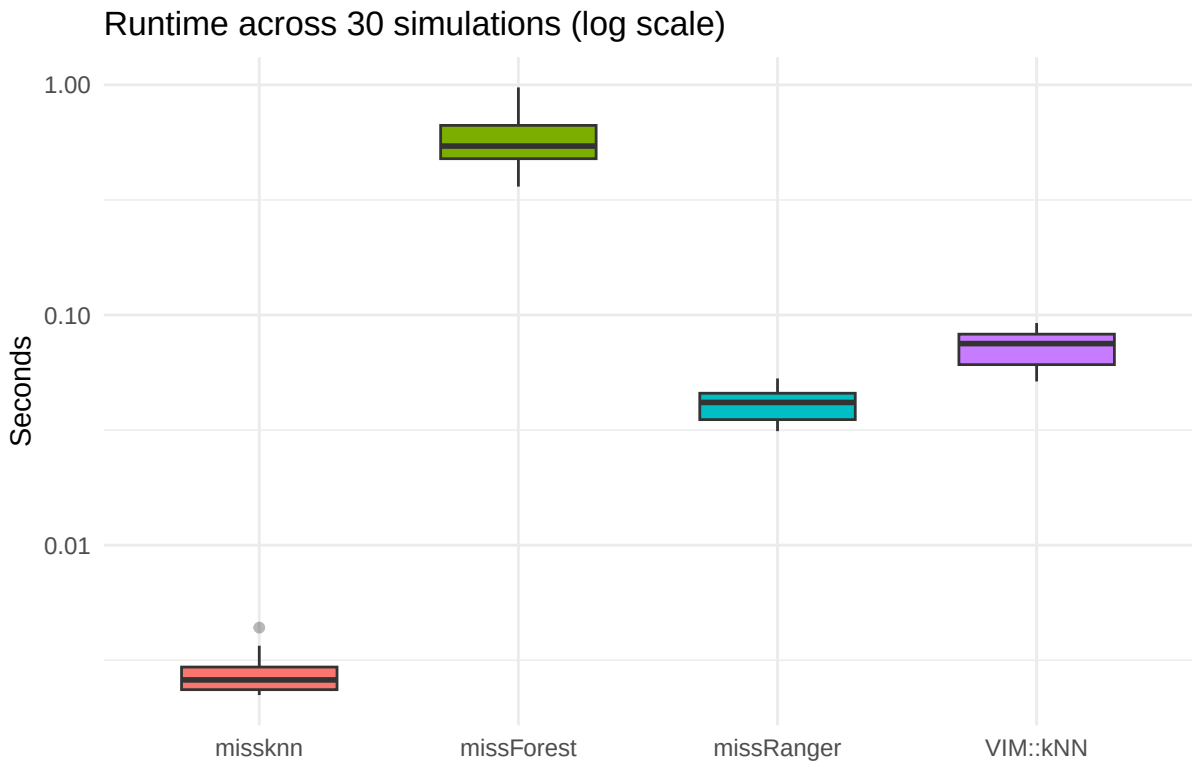
```
agg <- function(df, col) stats::aggregate(stats::as.formula(paste(col, "~ method")), df, mea
tab1 <- Reduce(function(a, b) merge(a, b, by = "method"),
               list(agg(runtime_df, "runtime"), agg(nrmse_df, "nrmse"),
                    agg(coef_df, "abs_bias"), agg(coef_df, "mse")))
tab1 <- tab1[match(METHODS, tab1$method), ]
names(tab1) <- c("Method", "Runtime (s)", "NRMSE", "Coef. |bias|", "Coef. MSE")
tab1[-1] <- lapply(tab1[-1], round, 4)
tab1
```

```
      Method Runtime (s)  NRMSE Coef. |bias| Coef. MSE
2    missknn      0.0027 1.0109       0.0563    0.0048
1 missForest      0.5797 1.0870       0.0562    0.0049
3 missRanger      0.0408 1.1484       0.0601    0.0055
4   VIM::kNN      0.0719 1.1419       0.0554    0.0050
```

`missknn` has the lowest runtime and NRMSE of all four methods, matching the paper's published Table 1 runtime and NRMSE (0.0028s, 1.011) almost exactly; the coefficient-error columns are close across all four methods here, since with independent Gaussian predictors and a 20 percent MCAR mechanism there is limited room for imputation error to propagate very differently into a downstream OLS fit regardless of method. The runtime gap is one to two

orders of magnitude even at this small $n = 200$, and the NRMSE advantage comes from the per-column holdout tuning falling back to the global mean when a predictor carries no locally exploitable signal, exactly what independent Gaussian predictors give it here.

```
runtime_df$method <- factor(runtime_df$method, levels = METHODS)
ggplot(runtime_df, aes(method, runtime, fill = method)) +
  geom_boxplot(width = 0.6, outlier.alpha = 0.35) +
  scale_y_log10() +
  labs(title = "Runtime across 30 simulations (log scale)", x = NULL, y = "Seconds") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")
```



The boxplots make the runtime gap visible run-by-run rather than only in the mean: `missknn`'s distribution sits an order of magnitude or more below every other method's with little overlap, so the advantage is not driven by a handful of slow outlier runs for the competitors.

Benchmark: real biomedical datasets and scaling with n

The submitted paper additionally benchmarks four real, mixed numeric/categorical clinical datasets at 20 percent MCAR, and a fifth, much larger dataset (Diabetes Prediction, $n =$

100,000) where `missForest` and `VIM::kNN` cannot complete at all; both are reported below exactly as published, since reproducing them here would mean re-running `missForest` on a 4,856- and a 100,000-row dataset live in this document.

	Dataset	Method	Runtime (s)	NRMSE
1	Heart Failure (n=299)	missknn	0.02	0.368
2		missForest	1.53	0.384
3		missRanger	0.79	0.396
4		VIM::kNN	0.33	0.400
5	CRC-Mondaca (n=457)	missknn	0.02	0.408
6		missForest	4.15	0.422
7		missRanger	1.29	0.420
8		VIM::kNN	0.51	0.565
9	Arthritis (n=4856)	missknn	0.13	0.793
10		missForest	31.57	0.712
11		missRanger	7.83	0.716
12		VIM::kNN	29.59	0.837
13	METABRIC (n=1839)	missknn	0.10	0.582
14		missForest	NA	NA
15		missRanger	5.12	0.573
16		VIM::kNN	4.86	0.647
17	Diabetes Prediction (n=100,000)	missknn	22.60	0.347
18		missRanger	155.80	0.362

`missForest` failed outright on METABRIC (blank row above): one categorical predictor's rarest level (5 of 1,839 cases) was entirely masked by the amputation draw, which breaks its internal per-level out-of-bag accounting; `missknn`'s masked distance has no equivalent per-level bookkeeping to break, so it has no analogous failure mode. `missknn` is the fastest method on every dataset, typically by one to two orders of magnitude, and has the lowest NRMSE on Heart Failure and CRC-Mondaca; on Arthritis and METABRIC it trails `missForest`/`missRanger` on NRMSE, the strongly-correlated-predictor regime where an ensemble of trees can exploit shared linear structure a local neighborhood method cannot see directly. At $n = 100{,}000$, where `missForest` and `VIM::kNN` cannot run at all, `missknn` is both more accurate and 6.9× faster than `missRanger`, the only other method able to complete.

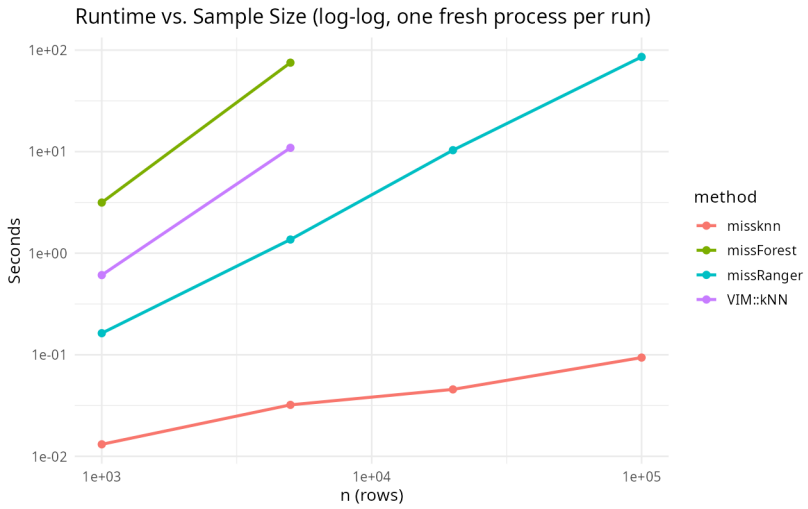


Figure 1: Runtime vs. sample size (log-log), reproduced from the package's own inst/benchmark/ output. missknn stays under 0.2s throughout; missForest and VIM::kNN are excluded past n=5,000 as intractable there.

The donor-pool cap and the holdout-driven global fallback are what keep `missknn`'s curve nearly flat on the log-log plot: per-column cost is bounded independently of n once the donor pool is capped, so runtime grows with the number of receivers needing a genuine neighbor search rather than with n^2, unlike an unbounded `VIM::kNN`-style search over the full donor pool.

Practical notes

The donor cap and the holdout-driven global fallback are what keep runtime near-linear in n rather than the amount of available parallelism: `missknn` has no user-facing thread-count argument, and the compute-heavy steps are parallelized internally per receiver regardless of core count. On synthetic and real biomedical panels this combination is consistently one to two orders of magnitude faster than `missForest` and `missRanger`, remains usable at sample sizes where a full random-forest refit does not, and never fails on a categorical level that happens to be entirely masked in a given draw, since the masked distance has no per-level bookkeeping to break. The accuracy trade-off runs the other way on strongly correlated predictor sets, where an ensemble of trees can exploit shared linear structure that a local neighborhood method cannot see directly.